

An Inverted Classroom Experience: Engaging Students in Architectural Thinking for Agile Projects

Jane Cleland-Huang
School of Computing
DePaul University
Chicago, IL. USA
jhuang@cs.depaul.edu

Muhammad Ali Babar
University of Adelaide
Adelaide, Australia
ali.babar@adelaide.edu.au

Mehdi Mirakhorli
School of Computing
DePaul University
Chicago, IL. USA
mehdi@cs.depaul.edu

ABSTRACT

This case study presents our experiences using architecturally savvy personnas in the classroom. The personas were used to help students analyze and prioritize architecturally significant requirements, and then to drive and evaluate architectural design. The activity was designed to equip students with a technique for integrating architectural thinking into the agile development process. We describe our learning goals and the activity, discuss student learning outcomes, lessons learned from running the activity, and propose an improved structuring. All materials including training videos, hand outs, and instructions are available online at <http://re.cs.depaul.edu/pedagogy/ASP>.

Categories and Subject Descriptors

D.2.1 [Software Engineering]: Requirements/ Specifications—*Elicitation methods (e.g., rapid prototyping, interviews, JAD)*; D.2.10 [Software Engineering]: Design—*Methodologies*; D.2.11 [Software Engineering]: Software Architectures

General Terms

Design

Keywords

Architecture, Pedagogy, Personas, Architecturally-Significant Requirements

1. INTRODUCTION

The prevalence of agile processes in the work place means that most students graduating with either masters or undergraduate degrees need to be equipped with skills to work in fast-paced, iterative, development environments. In such environments, architectural decisions are often made on the fly as the project proceeds [7]. Developers first create the simplest design needed to support current user stories, and

then incrementally add to and improve the design as the software evolves [24].

On the other hand, there is a need for more deliberate architectural thinking in early phases of the agile development process. Abrahamsson et al. [2] claim that “a healthy focus on architecture is not antithetic” to agility, and advocate the importance of finding the architectural sweet spot for a given project so that the emphasis on architecture is customized to the needs of the particular project. Boehm showed that, despite anecdotal wisdom, the cost of introducing change into an agile project does increase over time, unless an architecture has been designed to accommodate the kind of change anticipated in the system [8]. Similarly Beck asserted that architecture is just as important in XP (eXtreme Programming) projects as it is in any other project [7]. The question is therefore what kind of architectural practices should be adopted in an agile project.

Industrial architects attending the TwinPeaks workshop hosted at the 2013 IEEE International Requirements Engineering Conference stressed that even though architectural design may not be an official upfront activity in an agile project, a good architect will have a clear architectural vision for the solution in his or her head before embarking on development [19]. There are examples of this in practice. Scott Ambler proposed that architectures should be *sketched* in early phases of the project [3], while Denne and Cleland-Huang described the incremental funding method (IFM) in which architectures are planned upfront, but then delivered incrementally, as needed, to support user-required functionality [18].

Unfortunately the agile mantra of ‘no big upfront design’ is often used to justify a less than effective exploration of architectural concerns during early phases of the software development lifecycle, thereby increasing the likelihood of later costly refactoring efforts. Jan Bosch, a strong advocate for incremental design, acknowledges that it can lead to costly refactoring efforts as new requirements and new architectural concerns emerge [10]. However, he considers this an acceptable cost of doing business.

Given such mixed messages, students are often unable to reconcile lessons learned in requirements engineering and software architecture courses, with the fast-moving agile development environments in which they find themselves working. For example, requirements engineering courses may teach relatively traditional approaches that focus on upfront elicitation, analysis, and specification processes [23] or cover requirements negotiation techniques such as WinWin [9], while software architecture courses may stress the use of ar-

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from Permissions@acm.org.

Copyright is held by the author/owner(s). Publication rights licensed to ACM.

ICSE Companion’14, May 31 – June 7, 2014, Hyderabad, India
ACM 978-1-4503-2768-8/14/05
<http://dx.doi.org/10.1145/2591062.2591154>



Nielson
Stock Market Manager
of Operations

- ☐ -+30% transaction alert.
- ☐ Confirmation screen
- ☐ Cancel transaction
- ☐ -+30% transaction clearance
- ☐ Auto outgoing calls
- ☐ Monitor broker screens
- ☒ Halt error sales
- ☐ System demo
- ☒ On time transaction
- ☐ "what if" exchange

Nielson is married and has four children. After three years of working for the Australian Embassy in Japan, Nielson joined the Tokyo Stock Exchange as Manager of Operations. He has been working with TSE for seven years now. Nielson monitors the interactions of investment banks and brokerage firms with TSE, to ensure that transactions are completed correctly.

Nielson is a highly educated individual with two master degrees in Economics and Business Administration, as well as a PhD in finance. He thrives on hard work and executes it with great precision. He believes that there is no room for mistakes in his line of work.

My users stories:

1. I need the TSE system to deliver on-time transactions from investment banks.
2. I need the TSE system to inform me of any transactions (selling/buying) at 30% or less of market prices.
3. I need the TSE system to display broker information and contact numbers for any investment bank selling/buying at less than 30% of the market value.
4. I need to be able to immediately stop the sale of stocks when an error is discovered.

Figure 1: An AS-Persona created for a financial trading application (courtesy Niza Alahmadi)

architectural description languages [26] or front-loaded design activities such as Attribute Driven Design (ADD) [6]. Students are often not taught to integrate architectural thinking into the agile world of user stories, test-driven programming, and highly incremental delivery.

Our goal is to equip our students to think strategically about requirements and architectural design in an agile context. In particular, we wanted to expose them to the notion of *architecturally significant requirements* (ASRs) and their role in architectural design. Chen et. al, describe ASRs as requirements that “have a measurable impact on a software system’s architecture” [11]. They go on to say that ASRs are often hard to define and are therefore often expressed vaguely or hidden within other requirements. Franch et. al found that stakeholders often incorrectly assume that developers understand their unarticulated quality concerns, resulting in systems that fail to meet expectations [4].

To address our pedagogical goals we designed a classroom activity around the use of *architecturally savvy personas* (AS-Personas). The activity took place in an ‘inverted’ (or flipped) classroom. Gannod et. al explain that in an inverted classroom, learning activities that might normally be conducted as homework assignments, are performed during class time in the presence of the instructor, while the more passive activity of listening to lectures occurs outside of the classroom setting [20]. Our activity was designed to fully engage students in a hands-on learning process. The activity was conducted with 22 Master students at the IT University of Copenhagen in Denmark, and with 31 Graduate students at DePaul University in Chicago.

2. PERSONAS AND ARCHITECTURE

Before further describing our activity, we explain the notion of AS-Personas. Personas were first introduced for user interaction design by Alan Cooper in the mid 1990s [16]. In his now infamous book “The inmates are running the Asy-

lum” he recommended that software should be designed for archetypal users. Persona use has become increasingly popular over the years. A persona is typically given a name, and described in terms of psyche, background, emotions and attitudes [22, 21]. In most of the literature and practice, a persona is created following a fairly rigorous series of user analysis activities which include surveys and interviews, slicing of users into categories, data collection and analysis to determine whether candidate slices create distinguishable user groups, analysis of potential usage patterns within the groups, persona design, and ultimately the writing of scenarios which describe how the identified persona might interact with the system. These interactions are then used to drive and evaluate the user design of the system.

In our prior work, we have shown how personas can also be used to support the architectural design process [12, 15, 14]. While both user interaction design and architectural design are inherently creative activities, it is particularly important to get architectural design correct upfront. Modifying incorrect or inadequate architectural designs after-the-fact can be difficult and very costly. Using AS-Personas as an aid to understanding architectural concerns in early phases of the project can reduce missteps and minimize the need for late-stage costly refactorings.

We illustrate the notion of an AS-Persona with an example of an online trading system [25]. This system has numerous architectural concerns related to qualities such as security (e.g. confidentiality, authentication, and non-repudiation), performance (e.g. throughput and response time), reliability, scalability, usability (e.g. rigorous UI controls to help prevent incorrect transactions, fast cancellation of incorrect transactions if/when they do occur) etc. Balancing competing concerns, and delivering a system that satisfies stakeholders needs, can be very challenging. For example, a major blunder in the architectural design of the Mitsuho trading system [25], meant that in 2005 a trader accidentally traded 610,000 shares at 1 yen per share, instead of 1 share at 610,000 yen, and then upon realizing his mistake, was unable to cancel the transaction. The system designers had failed to fully understand and/or realize requirements for stopping incorrect transactions. More recently, the US government signed into law the Affordable Care Act in order to provide medical coverage for all Americans in the USA. Unfortunately the very public and highly politicized launching of the HealthCare.gov website led to high traffic volume which resulted in horrendous performance issues [13]. Contributing causes included a less than sufficient understanding of the ASRs and the subsequent deployment of an architecture which was incapable of scaling to meet demand.

To help prevent such problems we need to understand ASRs in advance. Our approach achieves this through the use of AS-Personas. We provide an example of an AS-Persona for the Mitsuho trading system in Figure 1. *Nielson* is a stock market manager. His major architectural concerns center around the timeliness and correctness of transactions. Furthermore, he is particularly concerned that erroneous transactions can be canceled ‘immediately’ upon request. Nielson’s user stories serve as placeholders for in-depth conversations that need to occur to determine more specific response time, transaction, and cancellation goals. For example, specific rules need to be established to determine under what conditions a transaction may be canceled, and what non-repudiation paths need to be established. However, the

Table 1: Representative selection of generic user stories for the Mechatronics System

R3 +++	JIT Tracing: MTS shall support the just-in-time generation of trace links (using information retrieval methods).	Rationale: The MTS system will be based on the concept of information retrieval. Various information retrieval algorithms will be used to "search" for related artifacts. Advanced concepts such as thesauri and ontologies will be used to improve results. (NOTE: current trace retrieval methods are typically able to return trace links at recall of 90% and precision of about 20-50%. (more...))
R4 +	Runtime Replacement of Trace Engine: The entire trace engine shall be replaceable at runtime.	Rationale: Trace retrieval includes a number of different processing steps such as (1) preprocessing (stemming, stopping, splitting camelCase, expanding acronyms), (2) core trace retrieval algorithms (VSM, LSA, LDA, voting), (3) different indexing techniques etc. The ideal combination depends on the data. The project manager shall have the option of replacing the trace engine. (more..)
R7 ++	Runtime Addition: New case tools must be added to the MTS at runtime.	Rationale: It is inconvenient to need to stop and start the system in order to add a new traceability artifact/case tool.
R11 ++	1000 users: The system needs to support 1000 simultaneous trace queries.	Rationale: The organization runs many simultaneous projects, and each project has many different users.
R19 ++	Response time: When a trace query is issued and the data is available the results shall be returned in less than 10 seconds.	Rationale: Trace users need trace results returned quickly so as to not interfere with the flow of their work.
R22 +++	Access Policies: Access to a model can be established according to policies defined for each case tool.	Rationale: Provides flexibility in the way access rights are managed.
R26 +++	Audit trail: An audit trail shall be maintained showing all data access.	Rationale: The data being traced represents critical IP to the company, therefore its access (especially by people not directly responsible for it) must be logged in an audit trail.
R28 +	24/7 data availability: Data must be available all the time even when a case tool is unavailable.	Rationale: Given the distributed nature of the MTS environment and the potential unavailability of certain case tools, trace data must still be available to support tasks.

persona clearly highlights a cohesive grouping of architecturally significant concerns. A general rule of thumb that has emerged for user interaction design is to identify 5-7 key personas per project. In our experiences of using AS-Personas for architectural design [15], we have also found that 5-7 personas can generally capture the main archetypal uses of the system.

Once created, personas are used to reason about the architectural design from the perspective of the users. In fact, the notion of using AS-Personas for architectural design emerged from a US\$2 Million development project in which we were engaged. Our project was agile-like. Requirements were captured as user stories, and architecture was sketched upfront and then delivered incrementally. In early phases of the project we realized that the stakeholders win conditions were rather complex and exhibited tradeoffs which were hard to balance. As a result, we created the notion of AS-personas in order to reason about ways to balance architectural concerns in order to optimize stakeholders' goals [15].

We found the AS-Personas to be beneficial in our project because they focused the architectural design around the expressed needs of our stakeholders and provided a vocabulary and context for discussing architectural concerns throughout the project.

3. PEDAGOGICAL GOALS

Given the practical success of our approach, we designed a classroom activity around the notion of AS-Personas to address the following pedagogical goals:

- Expose students to architectural thinking within the context of an agile development environment.
- Teach students to identify architecturally significant user stories and to explain their significance.

- Develop skills in using personas to drive architectural design.
- Augment existing design skills through engaging in a comprehensive architectural design activity.
- Equip students to critically evaluate the design from the personas' perspectives.
- Provide students with vocabulary and skills to communicate architectural design decisions with respect to stakeholders' concerns.

4. ACTIVITY DESCRIPTION

Our activity was initially designed to take place in a three hour time period. It involved the following steps: (1) preparation, (2) evaluating a set of user stories and identifying architecturally significant ones which would impact the architectural design, (3) constructing architecturally savvy personas, (4) designing the architectural solution, and finally (5) evaluating the overall experience.

4.1 Step 1: Preparation

To prepare students for the activity we provided introductory material about (1) the domain of the study, (2) general information about ASRs, and (3) an introduction to the use of AS-Personas. Video materials for all three of these areas are available on our website. As we have already discussed ASRs and AS-Personas, we focus here on describing the domain of the study.

Our design activity was conducted in the domain of Mechatronics Traceability [17, 12]. This domain was chosen because it represented a non-trivial problem that would be comprehensible to all students while also presenting the challenge of designing a system with capabilities to reach across multiple, distributed, heterogeneous data stores. Further-



Renee:
Requirements
Engineer.

Renee leads a team responsible for identifying relevant stakeholders and subject matter experts, and working with them to discover, specify, and prioritize requirements. Her work often involves traveling to meet with remote stakeholders. All stakeholders, whether local or remote, need to be able to view requirements. The remote geographical locations create occasional connectivity challenges. Renee will need to access the new MTS using her mobile device while traveling, and once development starts, she needs to track progress against requirements.



Devon
Senior
Developer

Devon is a senior developer assigned to the MTS system. He has 12 years of experience working at ASEC, three of them as senior developer. It is his job to head up a team of 10 programmers and to participate in the design, construction, and testing of MTS. He is worried about the problem of keeping the MTS system current as the organization and modeling environments evolve, and is concerned with the long-term challenge of integrating as yet unknown case tools into the system. He doesn't want to spend the rest of his career working on integration code.



Andy
Mechanical
Engineer.

Andy has worked as a Mechanical Engineer at ASEC for five years – primarily in the product development department of the medical division. He designs, simulates, and prototypes solutions for various parts of the medical device. He often interacts with electrical engineers. His team has to comply with numerous safety standards that impact the development of medical devices. It would help Andy if he could view relevant regulatory codes associated with models he is currently working on. Andy is a busy man and wants to make sure that trace results are returned quickly upon demand.



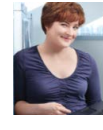
Sam
Security
Analyst

Sam is in charge of the infrastructure for single-sign-on (SSO) across the enterprise. He works to maintain the organizational firewall and has implemented several solutions for detecting security breaches. He is particularly concerned in ensuring that the MTS won't create massive additional overheads in terms of security and that it uses the already in place infrastructure. Ideally he would like to see a policy-based access control system – which uses the current SSO infrastructure. Given the sensitivity of the data, he insists that reliable audit controls be established.



Smita
Embedded
programmer

Smita is a software programmer for embedded devices. She has worked on numerous safety-critical projects for ASEC. She focuses much of her attention on ensuring that the software she writes satisfies design constraints of the system including power consumption, performance, and reliability. Her development cycle often includes formal specifications, model checking, simulations, programming, and then rigorous testing against the relevant requirements and design constraints. As model building is quite difficult, she wants to make sure that only her final, base lined models are made visible to other team members.



Celia
Safety Officer

Celia has worked as a safety officer for the past 12 years. She is a bulldog in ensuring that the hazard analysis is thorough, that fault analysis is conducted carefully, and that rigorous design constraints are specified and built into the product. Celia works with a team of risk analysts and safety officers. Her area of expertise is in standards such as DO-178B and FDA Guidelines for medical devices. She is frequently consulted by engineers and developers concerning compliance. She wants instant access to all data. For example, she wants to see latest test results the moment they have run, so that she can track problematic and risky parts of the system.

Figure 2: A Selection of Personas for the Mechatronics Traceability System

more, the team at DePaul had built a prototypical Mechatronics Traceability System (MTS) for Siemens, and so had a solid understanding of the design issues. The problem space deals with the integration of heterogeneous records stored in different formats across multiple geographical locations, and therefore has corollaries to the health-care domain, federated search, and many other better known problem spaces.

We presented the problem as enterprise level search in which engineers would perform text-based searches across various case tools and repositories in order to establish trace links upon demand. A trace link is defined by the Center of Excellence for Software Traceability (CoEST.org) as a link between a source artifact (such as a requirement) and a target artifact (such as a test case) [1]. Mechatronics systems typically include many different kinds of models, created and stored in a variety of case tools. While there could be separate models for requirements, software components, electrical components, thermodynamics, and mechanical components, they often interact to produce the final behavior of the mechatronics system. For example in a cyber-physical system, the software may control the behavior of a mechanical device, and therefore an engineer might like to trace from requirements to software code and also to the mechanical model. Therefore, the MTS should provide runtime, point-to-point traceability in the form of dynamic search, across a wide selection of case tools. Furthermore, these case tools are geographically distributed, have their own proprietary data formats, and either provide proprietary APIs for accessing the data, or allow access only through underlying data stores or data exports.

The students were instructed that the end goal of the activity was to sketch out an architectural design of the MTS.

4.2 Step 2: Analyze User Stories

Students were given 30 user stories describing the functionality of the MTS, eight of which are depicted in Table 1. The complete set of user stories is available for down-

load. The majority of these stories were deemed architecturally significant. For example, *R4*, which states that “The entire trace engine shall be replaceable at runtime” and also *R11* “The system needs to support 1000 simultaneous trace queries” are architecturally significant, while *R1* (not shown), which states that “MTS will include facilities to create and modify a Traceability Information Model”, is not architecturally significant. To help students further understand the domain, we prioritized requirements (+, ++, or ++++) and provided a rationale for each requirement. To make the activity more interesting, the user stories deliberately included redundant, hard-to-satisfy, and conflicting requirements. Students were told that they would ultimately need to reason about these trade-offs and inconsistencies and make decisions for how to balance the requirements.

Based on the previous definition of an ASR, students were instructed to work in small groups of 2-3 people to discuss the user stories, identify architecturally significant ones, and start to create a categorization scheme of various concerns i.e. performance, security etc.

4.3 Step 3: Create and Elaborate Personas

Because creating personas can be quite time consuming, and we wanted to focus attention on *using* them rather than creating them, we provided an initial outline for 12 candidate personas. As depicted in Figure 2, each one contained a picture and a brief description of the persona. Students were asked to select 4-6 of the persona outlines which, in their opinion, best represented distinct categories of stakeholders’ architectural concerns. They were then tasked with the job of assigning all high-priority architecturally significant user stories to personas in such a way that each persona represented a believable and cohesive perspective on the system. They were encouraged to change, augment, enrich, and merge personas and their interests. To help the process run more smoothly, we provided each of the requirements on an index card, and students assigned user stories to personas

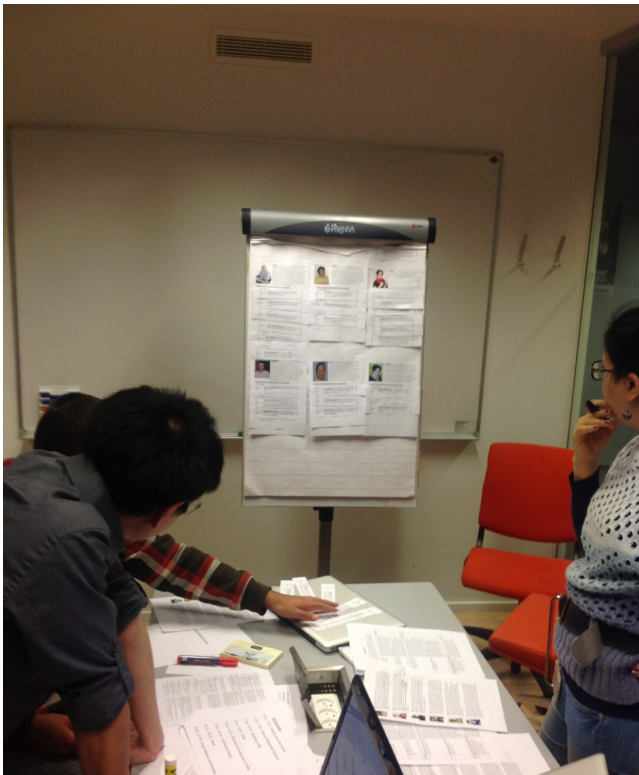


Figure 3: Students assign user stories to personas

by simply sticking the index cards onto the persona outline as shown in Figures 3 and 4.

4.4 Step 4: Sketch an Architectural Solution

The architectural design activity was based on Ambler's premise that architectural designs should be sketched out in early phases of the project [3]. Because there would not be time to design a complete architecture, each team was instructed to identify and focus upon three major architectural concerns. For example, concerns might include *authentication* and *access control* mechanisms, *runtime interchangeability of the trace engine*, *response time*, *extensibility* to accommodate new and varying case tools, etc.

To ensure that the students approached the design problem at sufficient depth, we seeded their architectural thinking with a series of questions. Clearly, some of these questions are more critical than others, and the activity involved determining which issues should be considered, and drive, initial architectural design. The questions included:

- Will the solution use a push or pull model?
- Will the solution use a centralized or distributed trace engine?
- How will data be extracted from heterogeneous case tools?
- How will the design be impacted if some case tools, and their associated data, are distributed remotely across organizational boundaries?
- Will the solution be shaped around any specific architectural style?
- Will the design allow the trace engine be replaced dynamically at runtime or only statically?

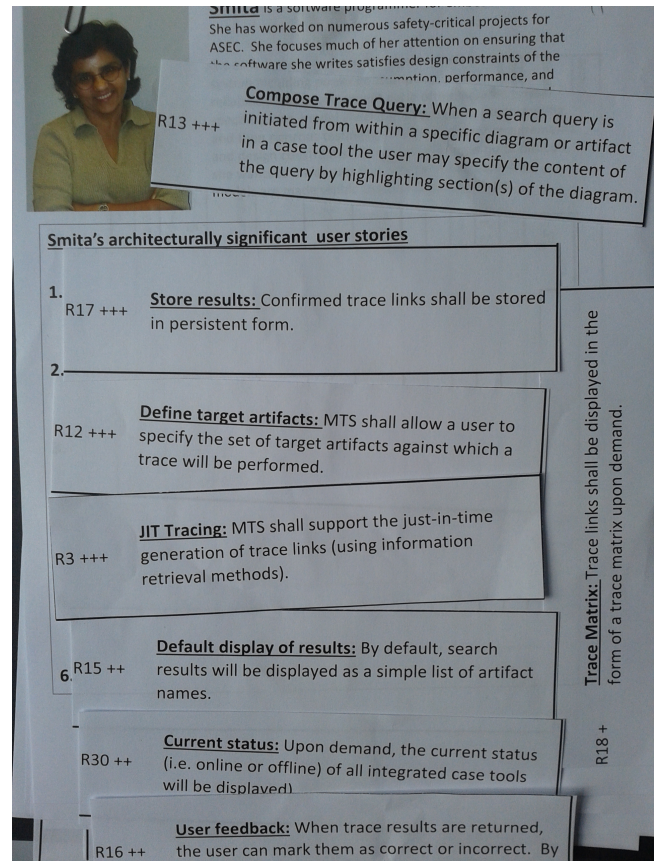


Figure 4: One of the produced personas

- How will data be exchanged across boundaries of the case tools?
- How will failures be handled?
- How will performance goals be met?
- How will new case tools be integrated into the system?
- Will the system scale up to support additional users? To what extent?

The answers to these questions should address the focused concerns of the created personas.

Students were instructed to provide a static view of the architecture (i.e. box and line), a deployment view, and any additional views (including basic lists) needed to communicate their architectural decisions [5].

4.5 Step 5: Evaluate the Solution

For pedagogical purposes we wanted the students to think very explicitly about the architectural decisions they made and the impact of those decisions upon their created personas. We therefore provided students with templates in which they could explicitly document their architectural decisions against specific concerns, and also map them against user stories and personas. An example of a template filled in by a team of students and depicting the architectural concern of *response time*, its four related user stories, architectural decisions, risks, and mappings to personas is shown in Figure 6.

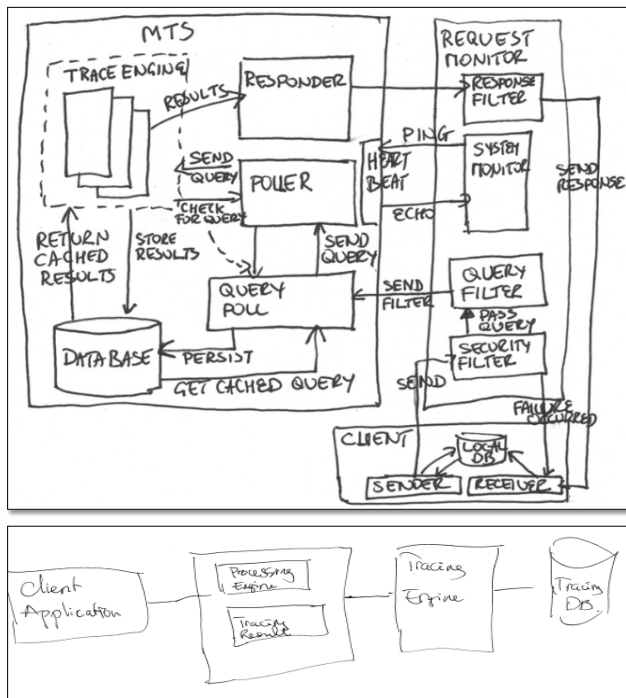


Figure 5: Two architectural sketches created by on-line (top) and in-class (bottom) students

5. AN EXPERIENCE REPORT

We conducted our activity with graduate students at two different universities. The first group (ITU) was comprised of 22 graduate students at the IT University of Copenhagen, Denmark. These students worked in six teams of three, and one team of four people. The second group included 31 graduate students at DePaul University, comprised of 18 in-class students working in six groups of three (DP1) and 13 online learners working individually (DP2). The majority of students at both locations work full-time in the IT industry in roles such as developer, enterprise architect, tester, and consultant. Adjustments were made for online learners who worked individually. In lieu of group-work, the instructor created an online forum in which the students could ask questions and post their final designs for discussion purposes.

5.1 Questionnaire

Following the activity, students were asked to complete the questionnaire shown below. All quantitative questions marked with * were answered on the following scale: (1=Not at all, 2 = Little, 3 = To some extent, 4 = Quite a lot, 5 = Very significant)

BEFORE THE ACTIVITY:

1. *How important is upfront architectural design in agile development processes? (Please explain your answer)

AFTER THE ACTIVITY:

2. What strategies did you use to select AS-personas?
3. * To what extent did the AS-personas help you organize the ASRs? (Please explain your answer.)

Quality Goal:	Fast, efficient search engine (performance)		Cella	Andy	Peter	Frank	Frank	Devon
Pertinent user stories:	R3	Just in Time Tracing	X					
	R19	10 Sec response time	X	X				
	R17	Store confirmed links			X			
	R2	Prevent ad-hoc trace links		X				
Architectural Decisions impacting this goal	AD 1.	Trace engine and indexes should be centralized						
	AD 2.	Search results must reflect actual data . Indexing must occur within 10 seconds after updates.						
	AD 3.	Trace engine must be designed to run on parallel processors to ensure scalability and perf.	✓	✓	✓			
	AD 4.	Access to trace data will be controlled by the central Trace server						
Risks	R 1.	Risk of out-of-sync indexes (data vs. index)						
	R 1.	Bottle neck, single point of failure.						
Personal Impacts	PI 1.	Frank may be dissatisfied with a centralized solution						
	PI 2.							

Figure 6: A student-filled template showing architectural decisions. (Note: For readability purposes we reproduced the students' work electronically).

4. * To what extent did the AS-personas help you to design your architectural solution? (Please explain your answer.)
5. * To what extent did the AS-personas help you to evaluate your architectural solution? (Please explain your answer.)
6. Do you think your architectural design was better because you used personas? (Yes/No) Please explain your answer.)
7. What did you like best about this activity?
8. What did you like least about this activity?
9. *How important is upfront architectural design in agile development processes? (Please explain your answer)

ITU students answered as a team (therefore we collected 7 data points), while students at DePaul all answered individually. Results from the quantitative part of the questionnaire are reported in Table 2.

5.2 Organizing ASRs

In response to question Q3 “to what extent did the AS-
personas help you organize the ASRs?” students gave an
overall rating of 3.7 (ITU), 3.6 (DP1), and 3.7 (DP2). Most
groups immediately eliminated low-priority requirements and
then filtered out non architecturally significant ones. They
then mapped the remaining user stories to personas and se-
lected the personas with the most critical and cohesive set
of user stories. Other groups selected the most interesting
personas first, i.e. those personas with more focused inter-
ests, and then mapped all architecturally significant, critical
requirements to the selected personas. In response to Q3,
students made comments such as “the personas helped us
identify what was important,” “the persona’s unique features
enable him/her to deeply understand clients’ requirements,”
“personas help determine fundamental requirements for the
architecture,” and “personas tell the requirements from the
user point of view. We accommodated all stakeholders by
using personas.” Some students were less positive. For ex-
ample, one student said that the helpful parts were “mostly
the technical aspects which we had to manually separate
from the persona’s characteristics.”

Table 2: Student reflection on the impact of Personas on Architecturally Significant Requirements

		CPH	DP1	DP2
Q1.	Before starting this assignment, how important do you consider upfront architectural design in agile development processes?		3.7	3.3
Q3.	To what extent did the personas help you organize the ASRs?	3.7	3.6	3.7
Q4.	To what extent did the personas help you to design your architectural solution?	3.1	3.2	3.2
Q5.	To what extent did the personas help you to evaluate your architectural solution?	2.3	3.2	3.3
Q9.	Following completion of the assignment, how important do you consider upfront architectural design in agile development processes?		4.05	3.5

5.3 Designing the Architecture

In response to question Q4 “to what extent did the AS-personas help you to design your architectural solution?” students gave an overall rating of 3.1 (ITU), 3.2 (DP1), and 3.2 (DP2). While students found AS-Personas less helpful for design purposes than for organizing ASRs, many of them did report positive aspects. For example, one student said that “Personas helped us reference ASRs by a human relatable name. For example ‘Andy needs... XYZ’ or ‘Andy will be happy with...’.” Another stated that “after knowing what the personas needed, we designed the architecture accordingly” and one said that he was able to “develop a preliminary architectural diagram that depicted how requirements are flowed down to different personas.” On the other hand, one student said that personas were pretty much a “non-factor” in the design. Yet another student noted that “each persona was so different (that) it was hard to determine the key requirements for design.” While this was meant as a critique, we perceived it as a positive factor that the student was forced to think about competing requirements during architectural design, especially as this is a real-world reality which is not always easy to reproduce in the classroom. We also anecdotally observed that the architectural sketches for the online students were vastly superior to those of in-class students. We believe that this was due to time constraints in the classroom.

5.4 Evaluating the Architecture

In response to question Q5 “to what extent did the AS-personas help you to design your architectural solution?” students gave an overall rating of 2.3 (ITU), 3.2 (DP1), and 3.3 (DP2). Student comments included “they gave me an idea about who will like the architectural decision,” “the description of the personas kept the design, requirements, and persona needs in synch,” and “we were able to check our design against the requirements designed by the personas.” On the other hand, one person said that once “we got the requirements aligned and organized we didn’t look at the personas again.” We do not yet have sufficient data to explain the discrepancy between the ITU and DPU groups.

5.5 Were AS-Personas Useful?

Q6 asked whether the team or individual felt that the architectural design was better because of the use of personas. Six of the ITU teams answered *yes* giving justifications such as “it helped us to specify the ASRs and to select the most important architectural concerns” or “it helped us manage the design decisions”, “prioritize ASRs,” or “relate the users

to the actual design.” The team that answered ‘maybe’ explained that the personas they created lacked focus. Out of the DP1 students who answered this question, 10 responded *yes* while 5 responded *no*. The reasons for the five negative responses were given as time limitations (3), over-focus on prioritization (1), and ‘didn’t add any value’ (1). For DP2 students 8 answered *yes* and 2 *no*.

5.6 The Best and Worst of the Exercise

Questions Q7 and Q8 asked the students what they liked best and least about this exercise. Perceived benefits included: focusing on risks associated with architectural decisions, feedback from the research assistant during the exercise, brainstorming ideas, the challenge of problem solving from a persona perspective, group interactions, persona driven design, and persona construction. Perceived problems included difficulty of understanding the impact of a design decision on a persona, unequal contributions from group members during the design activity, insufficient time, understanding the user stories, the need for clearer instructions, and the lack of some starting architectural designs.

5.7 Architectural Thinking in Agile Projects

The DePaul activity was conducted after the Copenhagen one, and we added an additional *before* and *after* question of “how important is upfront architectural design in agile development processes?” The average *before* rating was 3.70 and the *after* rating was 4.05 for DP1 students. Out of 18 students, 13 did not change their opinion, while 5 increased their rating. For example, a *before* comment by one student stated that the “architectural design is always changing in agile development” and that “spending lots of time at the start of the project might not be necessary;” however, by the end of the activity, the same student increased the rating by two points and stated that despite changes it is very important to “make sure that the architecture meets the requirements.” Another student who rated this question as a 4 in both the before and after question explained that “good architectural design will reduce development costs in the long run; however agile projects are more forgiving to changes when compared to waterfall modeled projects.” For DP2 students the average *before* score was 3.2 and the *after* score was 3.35. The student paired t-test with two tails was applied to the responses from all students. It returned a value of 0.02 which showed a significant difference in these ratings at $p=0.05$.

5.8 Threats to Validity

We chose not to set up our case study in a controlled way primarily because we wanted all of our students to benefit fully from the activity. The feedback is therefore rather qualitative in nature and it is certainly possible that students gave positive answers in order to please the instructors. However, the maturity of the students, the apparent freedom they felt to provide both positive and negative feedback, matched with their overall enthusiastic support for the activity suggests that the feedback reflects their true experiences.

We also considered evaluating the quality of the architectural design; however, we chose not to simply because the inclass students were quite rushed at the end of the activity, and as a result their architectural solutions lacked sufficient detail for this kind of analysis. We leave this assessment

to a future study conducted using the improved structuring described in section 5.9.

5.9 Conclusions

The activity achieved many of our pedagogical goals. Students were challenged to think about architecturally significant requirements, manage conflicting stakeholder goals, and sketch out an architectural solution which satisfied the goals within the context of an agile project. The classroom activity generated significant amounts of energy, and many students were enthused by the notion of a hands-on inverted classroom. Several of the online students mentioned that they wished they could have benefited more from the interactive nature of the activity, and so clearly we need to develop better approaches to engage online learners in inverted-classroom activities. We also noted that ITU students used separate conference rooms for the activity and therefore had less immediate interactive support from the instructor, while DePaul inclass students all worked within the same classroom. The evaluations and our own observations highlighted the benefits for students of seeing each other's personas and designs emerge, and also from the higher availability of the instructor and teaching assistant throughout the activity.

The major problem we faced was time limitation, which we feel could be successfully addressed by spreading the activity over several sessions. As a result we propose a new, more dispersed structuring of the activities:

1. **Session 1:** Introduce the notion of AS-personas (see AS-Persona video) (40 minutes)
2. **Session 2:** Introduce the domain i.e. Mechatronics Traceability (see MTS Video) (30 minutes). Perform the *User Story* activity and *create personas* (45 Minutes)
3. **Session 3:** Perform the architectural design activity (60 minutes).
4. **Take home:** Assign individual homework to fill in the templates and formalize the architectural documents.

We believe that this format will give students the time they need to think about and engage in the architectural design process. All materials needed to reproduce this activity can be found at <http://re.cs.depaul.edu/pedagogy/ASP> (case sensitive).

6. ACKNOWLEDGMENTS

This work was partially funded as a broader impact of National Science Foundation grant CCF-1218303.

7. REFERENCES

- [1] CoEST: Center of excellence for software traceability, <http://www.CoEST.org>.
- [2] P. Abrahamsson, M. Babar, and P. Kruchten. Agility and architecture: Can they coexist? *Software, IEEE*, 27(2):16–22, march-april 2010.
- [3] S. W. Ambler. Agile modeling: A brief overview. In *pUML*, pages 7–11, 2001.
- [4] D. Ameller, C. P. Ayala, J. Cabot, and X. Franch. Non-functional requirements in architectural decision making. *IEEE Software*, 30(2):61–67, 2013.
- [5] F. Bachman, L. Bass, P. Clements, D. Garlan, J. Ivers, R. Little, P. Merson, R. Nord, and J. Stafford. *Documenting Software Architectures: Views and Beyond (2nd Edition)*. Addison-Wesley, 2011.
- [6] L. Bass, P. Clements, and R. Kazman. *Software Architecture in Practice*. Addison Wesley, 2003.
- [7] K. Beck. *Extreme Programming Explained: Embrace Change*. Addison-Wesley, 2000.
- [8] B. Boehm, R. Turner, G. Booch, and A. Cockburn.
- [9] B. W. Boehm, A. Egyed, D. Port, A. Shah, J. Kwan, and R. J. Madachy. A stakeholder win-win approach to software engineering education. *Ann. Software Eng.*, 6:295–321, 1998.
- [10] J. Bosch and D. Dvorak. Traversing the twin peaks. *IEEE Software*, 2012.
- [11] L. Chen, M. A. Babar, and B. Nuseibeh. Characterizing architecturally significant requirements. *IEEE Software*, 30(2):38–45, 2013.
- [12] J. Cleland-Huang. Meet elaine: A persona-driven approach to exploring architecturally significant requirements. *IEEE Software*, 30(4):18–21, 2013.
- [13] J. Cleland-Huang. Don't fire the architect! where were the requirements? *IEEE Software*, 31(2):27–29, 2014.
- [14] J. Cleland-Huang, A. Czauderna, and E. Keenan. A persona-based approach for exploring architecturally significant requirements in agile projects. In *REFSQ*, pages 18–33, 2013.
- [15] J. Cleland-Huang, A. Czauderna, and M. Mirakhorli. Driving architectural design and preservation from a persona perspective in agile projects. In M. Ali Babar et. al, editor, *Agile Software Arch.: Aligning Agile Processes and Software Architectures*. Elsevier, 2013.
- [16] A. Cooper. The inmates are running the asylum. In *Software-Ergonomie*, page 17, 1999.
- [17] A. Czauderna, J. Cleland-Huang, and B. Berenbach. Just-in-time mechatronics traceability. *Siemens White Paper*, pages 1–56, 2010.
- [18] M. Denne and J. Cleland-Huang. The incremental funding method: Data-driven software development. *IEEE Software*, 21(3):39–47, 2004.
- [19] M. Galster, M. Mirakhorli, J. Cleland-Huang, J. E. Burge, X. Franch, R. Roshandel, and P. Avgeriou. Views on software engineering from the twin peaks of requirements and architecture. *ACM SIGSOFT Software Engineering Notes*, 38(5):40–42, 2013.
- [20] G. C. Gannod, J. E. Burge, and M. T. Helmick. Using the inverted classroom to teach software engineering. In *ICSE*, pages 777–786, 2008.
- [21] L. Nielsen. *Personas - User Focused Design*, volume 15 of *Human-Computer Interaction Series*. Springer, 2013.
- [22] C. Putnam, B. E. Kolko, and S. Wood. Communicating about users in ictd: leveraging hci personas. In *ICTD*, pages 338–349, 2012.
- [23] S. Robertson and J. Robertson. *Mastering the Requirements Process*. Addison Wesley, 2006.
- [24] J. Shore and S. Warden. *The Art of Agile*. O'Reilly, 2007.
- [25] T. Tamai. Social impact of information system failures. *IEEE Computer*, 42(6):58–65, 2009.
- [26] R. N. Taylor, N. Medvidovic, and E. M. Dashofy. *Software Architecture - Foundations, Theory, and Practice*. Wiley, 2010.